Princeton University

# ORF 363/COS 323
# Final Exam, Spring 2024

May 9 (9:00AM) to May 11 (9:00AM), 2024

*Instructor:*
Ioannis Akrotirianakis

*AIs:*
Thomas Bosancic, Sofiia Shvaiko

1. Please write your name on the first page of your solutions. Next to it, please write out and sign the following pledge: "I pledge my honor that I have not violated the Honor Code or the rules specified by the instructor during this examination."
2. The exam is not to be discussed with *anyone* except possibly the instructors and the AIs. You can only ask *clarification questions*, and only as *public* (and preferably non-anonymous) questions on Ed Discussion. No emails.
3. You are allowed to consult the lecture notes, your own notes, the reference books of the course as indicated on the syllabus, the problem sets and their solutions (yours and ours), the midterm and its solutions (yours and ours), the practice midterm and final exams and their solutions, and all Ed Discussion posts, but *nothing else*. You can only use the Internet in case you run into problems related to software.
4. You may refer to facts proven in the notes or problem sets without reproving them.
5. For all computational problems, include your code. The output that you present should come from your code.
6. Unless you have been granted an extension because of overlapping finals, the exam is to be turned in on Saturday (May 11, 2024) at 9 AM EDT on Gradescope as a single PDF file. You are free to write your solutions on paper or on a tablet, or to type them up. Only the latest version submitted before your deadline will be graded.
7. Each question has 25 points. You need to justify your answers to receive full credit.

**Problem 1: Convexity**

(a) [13 points] Consider the function $g : [a, b] \to \mathbb{R}$ with $g(x) = -(1 - e^{\gamma(x-a)})(1 - e^{\gamma(b-x)})$, for some $a, b \in \mathbb{R}$ and $\gamma > 0$.

(i) Show that the function $g$ has a critical point at the middle point of the interval $[a, b]$.

(ii) Show that the function $g$ is convex. Is it strictly convex?

(iii) Based on your discoveries in (ii), characterize the critical point you found in (i).

(b) [12 points] Consider the convex function $f : \mathbb{R} \to \mathbb{R}$ and any $a, b \in$ **dom** $f$, with $a < b$.

(i) Show that for every $c \in [a, b]$ the following inequality holds

$$f(c) \leq \frac{b-c}{b-a} f(a) + \frac{c-a}{b-a} f(b)$$

(ii) Show that the following inequalities hold

$$\frac{f(c) - f(a)}{c - a} \leq \frac{f(b) - f(a)}{b - a} \leq \frac{f(b) - f(c)}{b - c}$$

**Solution**

(a) We note that the function has some interesting structure.

(i) the first derivative of $f$ is

$$f'(x) = \gamma e^{\gamma(x-a)}(1 - e^{\gamma(b-x)}) + (e^{\gamma(x-a)} - 1)\gamma e^{\gamma(b-x)} = \gamma e^{\gamma(x-a)} - \gamma e^{\gamma(b-x)}$$

Using FONC we have:

$$f'(x) = 0 \implies \gamma e^{\gamma(x-a)} - \gamma e^{\gamma(b-x)} = 0 \implies x - a = b - x \implies x = (b + a)/2$$

(ii) The second derivative is

$$f''(x) = \gamma^2 e^{\gamma(x-a)} + \gamma^2 e^{\gamma(b-x)}$$

which is positive for all $x \in [a, b]$. Hence $f$ is strictly convex function.

(iii) Since the function $g$ is strictly convex, the midpoint of the interval $[a, b]$ is the unique minimum (global minimum).

(b) We will use the definition of a convex function.

(i) Since $f$ is a convex function the Jensen inequality is satisfied for any $[a, b] \in \mathrm{dom} f$ and $\lambda \in [0, 1]$:

$$f(\lambda a + (1-\lambda)b) \leq \lambda f(a) + (1-\lambda) f(b) \tag{1}$$

Now for some random $c \in [a, b]$ define $\lambda = \frac{b-c}{b-a}$, which is in $[0, 1]$. The convex combination $\lambda a + (1-\lambda)b$ becomes

$$\lambda a + (1-\lambda)b = \frac{b-c}{b-a}a + (1 - \frac{b-c}{b-a})b = c \tag{2}$$

In addition, the right hand side of (1) becomes

$$\lambda f(a) + (1-\lambda) f(b) = \frac{b-c}{b-a} f(a) + (1 - \frac{b-c}{b-a}) f(b) = \frac{b-c}{b-a} f(a) + \frac{c-a}{b-a} f(b) \tag{3}$$

Substituting (2) and (3) into (1) we obtain

$$f(c) \leq \frac{b-c}{b-a} f(a) + \frac{c-a}{b-a} f(b) \tag{4}$$

(ii) We will use the result from part (i).

We first show

$$\frac{f(c) - f(a)}{c-a} \leq \frac{f(b) - f(a)}{b-a} \tag{5}$$

Subtracting $f(a)$ from both sides of (4) and doing the algebra, we have

$$f(c) - f(a) \leq \frac{b-c}{b-a} f(a) + \frac{c-a}{b-a} f(b) - f(a) \quad \Longrightarrow$$

$$f(c) - f(a) \leq \frac{c-a}{b-a}(f(b) - f(a)) \tag{6}$$

Dividing by $(c-a)$ both sides we obtain (5).

We now show that

$$\frac{f(b) - f(a)}{b-a} \leq \frac{f(b) - f(c)}{b-c} \tag{7}$$

We subtract $f(b)$ from both sides of (4) and doing the algebra, we have

$$f(c) - f(b) \leq \frac{b-c}{b-a} f(a) + \frac{c-a}{b-a} f(b) - f(b) \quad \Longrightarrow$$

$$f(c) - f(b) \leq \frac{b-c}{b-a}(f(a) - f(b))$$

Dividing by $(b-c)$ we obtain

$$\frac{f(c) - f(b)}{b-c} \leq \frac{f(a) - f(b)}{b-a}$$

Multiplying both sides of the above inequality by $-1$ we obtain (7).

**Problem 2: Nonlinear least squares and bacteria growth prediction**

Graduate students Alice and John, specializing in bacteriology[1], were tasked by their advisor, Dr. McCell, with developing a model to predict population growth in a new type of bacterium. Dr. McCell provided them with the dataset shown in Table 1, which details the population size of some samples she has just received from one of her collaborators.

| Time (t) | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Size (s) | 70 | 73 | 78 | 85 | 91 | 94 | 96 | 92 | 90 | 87 | 90 | 93 | 91 | 89 | 88 | 90 | 92 | 89 |

Table 1: Bacterial population growth during 18 hours

Upon examining and plotting the data, they proposed the following two candidate models

$$\text{Alice's model:} \qquad f_A(c;t) = c_2(1-\exp(-c_1 t)) + c_0 \exp(-c_1 t) - c_3 \cos(t),$$
$$\text{John's model:} \qquad f_J(d;t) = d_2(1-\exp(-d_1 t)) + d_0 \exp(-d_1 t),$$

where $c = (c_0, c_1, c_2, c_3)^T$ and $d = (d_0, d_1, d_2)^T$ are the parameter vectors for each model. Your task is to help Alice and John determine the best parameter values of their models.

(a) [8 points] Formulate the problems as nonlinear regression problems. Clearly identify the decision variables and the data in each model.

(b) [9 points] Implement the Gauss-Newton method to compute the optimal parameter values for both models using Matlab or Python. For Alice's model use the starting point: $c^0 = (50, 1, 100, 5)^T$, and for John's model use the starting point $d^0 = (50, 1, 100)^T$. Terminate your code when it reaches 100 iterations or when the difference between two consecutive iterations is less than $10^{-6}$ (for example, for model $f_A$ we can represent this difference as: $\|c^k - c^{k-1}\|_2 < 10^{-6}$, where $k$ is the iteration counter).

(c) [8 points] Plot the dataset and the two models on the same graph. Which model describes (fits) the dataset better? Briefly justify your answer by comparing the optimal objective function values of the two optimization problems.

**Solution:**

[1]Bacteriology is a branch of biology that studies the morphology, genetics, biochemistry, and ecology of bacteria.

(a) We have been given data points of the form $(t_i, s_i)$ where $s_i$ is the size of the population at time $t_i$. There are $m = 18$ data points given in Table 1. The unknown decision variables (parameters) in Alice's model are $c = (c_0, c_1, c_2, c_3)^T \in \mathbb{R}^4$ and the unknown decision variables (parameters) in John's model are $d = (d_0, d_1, d_2)^T \in \mathbb{R}^3$. Both models have two terms in common. As we will see later, the additional term $-c_3 \cos(t)$ in Alice's model helps it to capture variations in data better than John's model.

Our ultimate aim is to compute the unknown parameters of each model so that each fits the dataset as best as possible. In other words we want to minimize the deviations between the true values (i.e., the true population sizes $s_i$) and the values predicted by each of the models. The deviations (aka, residuals) of Alice's predicted model are defined as

$$G_i(c; t) = (s_i - f_A(c; t_i)), i = 1, \ldots, 18$$

Similarly for John's model the deviations are defined as:

$$g_i(d; t) = (s_i - f_J(d; t_i)), i = 1, \ldots, 18$$

Then the nonlinear least squares problems for Alice and John can be formulated as the following optimization problems

$$\min_{c \in \mathbb{R}^4} \frac{1}{2} \sum_{i=1}^{18} G_i^2(c; t) = \frac{1}{2} \sum_{i=1}^{18} (s_i - f_A(c; t_i))^2 \tag{8}$$

and

$$\min_{d \in \mathbb{R}^3} \frac{1}{2} \sum_{i=1}^{18} g_i^2(d; t) = \frac{1}{2} \sum_{i=1}^{18} (s_i - f_J(d; t_i))^2 \tag{9}$$

(b) You can use either Matlab or Python to implement the Gauss-Newton method.

**Update:** It was reported by several students that the starting point provided was unstable resulting in nearly singular matrices during the execution of the Gauss-Newton method. We have modified the solutions to cover those cases.

The main idea (which was implemented by several students) is to modify the matrix $J * (x_k)^T J(x_k)$ when it becomes nearly singular (e.g., when some of its eigenvalues become small). This can be achieved by adding a small enough multiple of the identity matrix and then considering the modified matrix $(J * (x_k)^T J(x_k) + \lambda I)$, where $\lambda > 0$ is a small number.

An alternative approach considered by the students was to bypass the nearly singular matrices by using the least squares method (`lstsq` in Python and `lsqr` in Matlab),

or the pseudoinverse (`pinv`) to solve the system of linear equations. Then they used the solution as the search direction which made the Gauss-Newton converge. Some students also changed the starting point to another one that was making the method converge (e.g., $c^0 = (50, 0.75, 100, 5)$ and $d^0 = (50, 0.75, 100)$).

The main aim of this exam problem was to test knowledge of the basic steps of the Gauss-Newton method (without the possibility of nearly singular matrices). The test on our computers did not demonstrate any numerical instabilities. Solutions that applied those steps correctly and converged will receive full grades.

**Matlab implementation:**

We first define the functions that compute the objectives and the Jacobian matrices of the two models.

```
%%%% Alice's functions
function result = Alice(x, t)
    result = x(3) .* (1 - exp(-x(2) .* t)) + x(1) .* exp(-x(2) .* t)
       - x(4) .* cos(t);
end

function gradient = grad_Alice(x, t)
    common = exp(-x(2) .* t);
    dfdx0 = common;
    dfdx1 = (x(3) - x(1)) .* t .* common;
    dfdx2 = 1 - common;
    dfdx3 = -cos(t);
    gradient = [dfdx0; dfdx1; dfdx2; dfdx3];
end

function jacobian = jacobian_Alice(x, t)
    g = grad_Alice(x, t);
    jacobian = [-g(1,:); -g(2,:); -g(3,:); -g(4,:)];
end


%%%% John's functions
function result = John(x, t)
    result = x(3) .* (1 - exp(-x(2) .* t)) + x(1) .* exp(-x(2) .* t);
end

function gradient = grad_John(x, t)
```

```
    common = exp(-x(2) .* t);
    dfdx0 = common;
    dfdx1 = (x(3) - x(1)) .* t .* common;
    dfdx2 = 1 - common;
    gradient = [dfdx0; dfdx1; dfdx2];
end

function jacobian = jacobian_John(x, t)
    g = grad_John(x, t);
    jacobian = [-g(1,:); -g(2,:); -g(3,:)];
end


%%%% residuals are common
function residual_vec = residual(y, x, t, f)
    test = f(x, t);
    residual_vec = y - test;
    residual_vec = residual_vec'; % need to convert a row vector to
       column vector - Matlab characteristic...
end
```

Now that we have defined these functions we can use them to compute the Gauss-Newton iterations. **Note that**, according to Matlab's syntax, we should always place the functions we just defined, at the end of the following listing (see the comments at the end of the code below).

```
% Data from Table 1
t = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18];
s = [70, 73, 78, 85, 91, 94, 96, 92, 90, 87, 90, 93, 91, 89, 88, 90,
   92, 89];

% Algorithmic parameters - convergence tolerance and maximum number
   of iterations
TOL = 1.0e-6;
MAX_ITER = 100;


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%% Alice's model computations %%%%%%

% Initial or starting point
ck = [50, 1, 100, 5];
```

```
%%% alternative starting point;
%%% it does not produce iterates with nearly singular matrices, and
   does not require any modifications
% ck = [50, 0.75, 100, 5];

% Gauss-Newton iterations for Alice's model
for k = 1:MAX_ITER
    c_prev = ck;
    gk = residual(s, ck, t, @Alice);
    Jk = jacobian_Alice(ck, t);
    JJt = Jk * Jk';
    rhs = Jk * gk;

    %%% check if you have a strongly invertible matrix
    % find the min eigenvalue
    [eigvals, eigvecs] = eig(JJt);
    min_eigval = min(eigvals);

    lambd = 0.0;
    if min_eigval <= 1.e-1
        % add a small purturbation, when you find a small eigenvalue
        % note: the values 1.e-1 and 0.005 are problem dependent and
           different values may produce different results
        lambd = 0.005;
    end

    % dir = pinv(JJt + lambd*eye(4)) * rhs;
    dir = (JJt + lambd*eye(4)) \ rhs;
    dir = dir';
    ck = ck - dir;

    % Check if converged
    term_crit = norm(c_prev - ck);
    if term_crit < TOL
        break;
    end
end

disp('Optimal parameters in Alices model:')
disp(ck);
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%% John's model computations %%%%%%

% Initial or starting point
dk = [50, 1, 100];

%%% alternative starting point;
%%% it does not produce iterates with nearly singular matrices, and
   does not require any modifications
% dk = [50, 0.75, 100];

% Gauss-Newton iterations for Alice's model
for k = 1:MAX_ITER
    d_prev = dk;
    gk = residual(s, dk, t, @John);
    Jk = jacobian_John(dk, t);
    JJt = Jk * Jk';
    rhs = Jk * gk;

    %%% check if you have a strongly invertible matrix
    % find the min eigenvalue
    [eigvals, eigvecs] = eig(JJt);
    min_eigval = min(eigvals);

    lambd = 0.0;
    if min_eigval <= 1.e-1
        % add a small purturbation, when you find a small eigenvalue
        % note: the values 1.e-1 and 0.005 are problem dependent and
           different values may produce different results
        lambd = 0.005;
    end

    % dir = pinv(JJt + lambd*eye(3)) * rhs;
    dir = (JJt + lambd*eye(3)) \ rhs;
    dir = dir';
    dk = dk - dir;

    % Check if converged
    term_crit = norm(d_prev - dk);
    if term_crit < TOL
        break;
    end
```

```
end

disp(newline);
disp('Optimal parameters in Johns model: ')
disp(dk);
```

## Python implementation:

```
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
import scipy.linalg as la

# Data from Table 1
t = np.array([ 1,  2,  3,  4,  5,  6,  7,  8, 9,  10, 11, 12, 13, 14,
    15, 16, 17, 18])
s = np.array([70, 73, 78, 85, 91, 94, 96, 92, 90, 87, 90, 93, 91, 89,
    88, 90, 92, 89])

# Algorithmic parameters -
# convergence tolerance and maximum number of iterations
TOL = 1.0e-6
MAX_ITER = 100

##################################
### Alice's model computations ###

def Alice(x,t):
    return x[2] * (1 - np.exp(-x[1] * t)) + x[0] * np.exp(-x[1] * t)
       - x[3] * np.cos(t)

def grad_Alice(x,t):
    common = np.exp(-x[1] * t)
    dfdx0 = common
    dfdx1 = (x[2] + x[0]) * t * common
    dfdx2 = 1 - common
    dfdx3 = -np.cos(t)
    return np.array([dfdx0, dfdx1, dfdx2, dfdx3])

def jacobian_Alice(x,t):
    g = grad_Alice(x,t)
    return np.array([-g[0], -g[1], -g[2], -g[3]])
```

```python
def residual(x,t,f):
    test = f(x,t)
    return s - test


# initial or starting point
ck = np.array([50,1,100,5])

# Gauss-Newton iterations for Alice's model
for k in range(MAX_ITER):
    c_prev = ck.copy()
    gk = residual(ck,t,Alice)
    Jk = jacobian_Alice(ck,t)
    JJt = Jk @ Jk.T
    rhs = Jk @ gk
    ck = ck - np.linalg.solve(JJt, rhs)

    # check if you converged
    termination_criterion = la.norm(c_prev-ck)
    # print(i, termination_criterion)
    if termination_criterion < TOL:
        break

print("\nAlice's optimal parameters: \n", ck)


##################################
### John's model computations ###

n_param = 3

def John(x,t):
    res_local = x[2] * (1 - np.exp(-x[1] * t)) + x[0] * np.exp(-x[1]
        * t)
    return res_local

def grad_John(x,t):
    common = np.exp(-x[1] * t)
    dfdx0 = common
    dfdx1 = (x[2] + x[0]) * t * common
    dfdx2 = 1 - common
```

```
    return np.array([dfdx0, dfdx1, dfdx2])

def jacobian_John(x,t):
    g = grad_John(x,t)
    return np.array([-g[0], -g[1], -g[2]])


# initial or starting point
dk = np.array([50,1,100])

# Gauss-Newton iterations for Alice's model
for k in range(MAX_ITER):
    dprev = dk.copy()
    gk = residual(dk,t,John)
    Jk = jacobian_John(dk,t)
    JJt = Jk @ Jk.T
    rhs = Jk @ gk
    dk = dk - np.linalg.solve(JJt, rhs)

    # check if you converged
    termination_criterion = la.norm(dprev-dk)
    #print(i, termination_criterion)
    if termination_criterion < TOL:
        break

print("\nJohn's optimal parameters: \n", dk)
```

(c) The plot of Alice's and John's models together with Dr. McCell's data are shown in Figure 1. As can be seen Alice's model describes Dr. McCell's data much better than John's. Alice's model is able to capture the preriodicity of the data after the $t = 8$

The objective functions of the two optimization models (8) and (9) represent the squared deviations between the true values and the predicted values. Alice's model objective value is 80.7137 and John's objective value is 153.1461. Since Alice's model has lower squared deviations her model fits the data better than John's. Here is the Matlab and Python code to compute the deviations for each model.

**Matlab implementation:**

```
%%% Calculate the fitness of each model %%%

% Alice's model
```

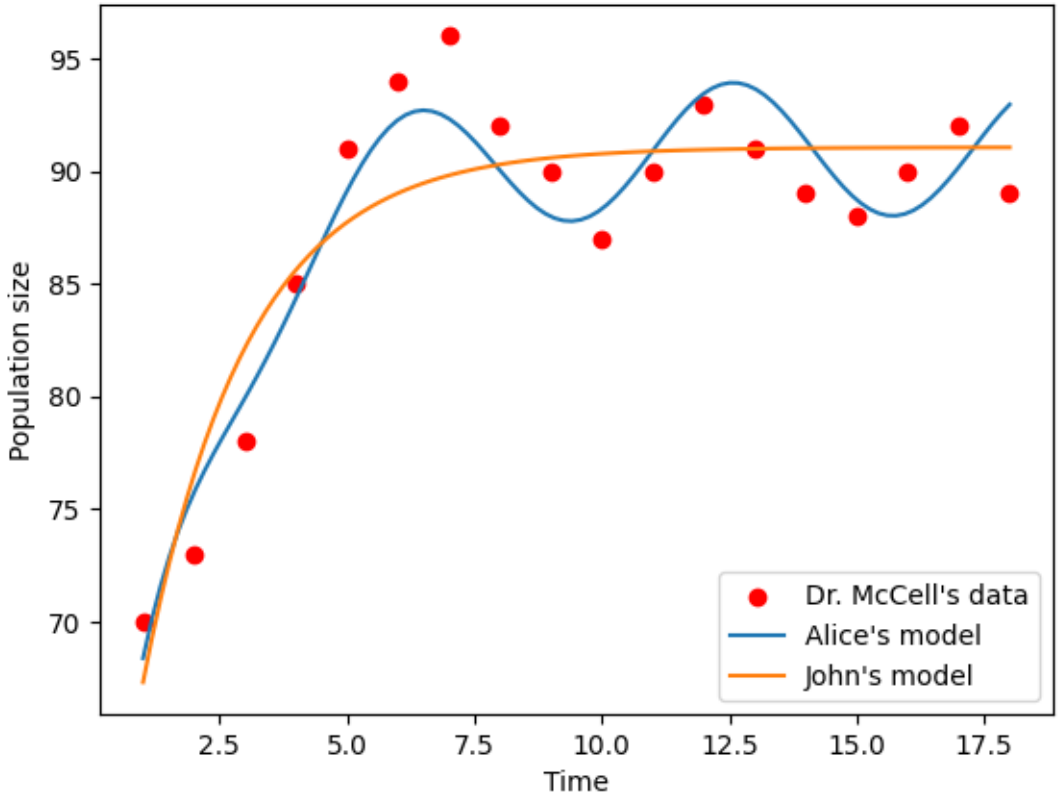


Figure 1: Comparison of the two models and how well they fit the data

```
pred_vec_A = Alice(ck, t);
sq_deviation_A = (s - pred_vec_A).^2;
fitness_A = 0.5 * sum(sq_deviation_A);
fprintf("\nAlices's fit: %f\n", fitness_A);

% John's model
pred_vec_J = John(dk, t);
sq_deviation_J = (s - pred_vec_J).^2;
fitness_J = 0.5 * sum(sq_deviation_J);
fprintf("\nJohn's fit: %f\n", fitness_J);



%%%% Plot the data and the model %%%
```

```
% Dr. McCell's data points
scatter(t, s, 'ro', 'MarkerFaceColor', 'r');
hold on;

% Define the x-axis (time axis of 100 points)
time_axis = linspace(1, length(t), 100);

% Plot Alice's model
predictions_A = Alice(ck, time_axis);
%plot(time_axis, predictions_A, 'b-', 'label', "Alice's model");
plot(time_axis, predictions_A, 'b-', 'LineWidth', 1.5);
hold on;  % Keep the plot open for adding more lines

% Plot John's model
predictions_J = John(dk, time_axis);
%plot(time_axis, predictions_J, 'r-', 'label', "John's model");
plot(time_axis, predictions_J, 'k-', 'LineWidth', 1.5);

% Labels and legend
xlabel('Time');
ylabel('Population size');
legend('McCells model', 'Alices model','Johns model', 'Location','
   southeast');

% Display the plot
grid on;  % Add grid lines for better visualization
title('Population Size Over Time');
```

## Python implementation:

```
### calculate the fitness of each model ###

pred_vec_A = Alice(ck,t)
sq_deviation_A = (s-pred_vec_A)**2
fitness_A = 0.5*np.sum(sq_deviation_A)
print("\nAlices's fit: ", fitness_A)
# fitness_A = 0.5 * 80.7137 = 40.3668

pred_vec_J = John(dk,t)
sq_deviation_J = (s-pred_vec_J)**2
fitness_J = 0.5*np.sum(sq_deviation_J)
print("\nJohn's fit: ", fitness_J)
```

```
# fitness_J = 0.5 * 153.1461 = 76.5730


### plot the data and the models ###

plt.scatter(t,s, c='red', label="Dr. McCell's data")

# define the x axis (time axis of 100 points)
time_axis = np.linspace(1,len(t),100)

# plot Alice's model
predictions_A = Alice(ck,time_axis)
plt.plot(time_axis,predictions_A, label="Alice's model")
#plt.show()

# plot John's model
predictions_J = John(dk,time_axis)
plt.plot(time_axis,predictions_J, label="John's model")

plt.xlabel('Time')
plt.ylabel('Population size')
plt.legend()
plt.show()
```

**Problem 3: Zoe's fun-maximization problem**

Zoe has recently completed her first year of graduate school in London, UK. As she reflects on this milestone, she realizes that life isn't solely about equations and coding; it's also about discovering the beauty of the city and participating in its numerous cultural events. With her expenses neatly organized, she's now prepared to embark on a quest to optimize her fun time and make the most of her hard-earned money.

**The Ledger of Expenses:** Zoe has meticulously compiled her financial data from the past year. Here's what she's got:

- Living expenses: Every month, she allocates £550 for the essentials—food, toiletries, and the occasional late-night snack attack.
- Rent: Her cozy studio costs her £630 each month. It's her sanctuary, complete with a window seat for daydreaming.
- Telephone Connection: She is charged £135 by the phone company every two months, on even months.
- Electricity and Gas: Every six months (the 6th and the 12th month), Zoe faces an £850 bill for keeping her apartment cool in summer and warm in the winter.
- Transportation: To move around London she needs to buy a pass for the London Underground costing £340 per month.
- Taxes: Every four months, she needs to pay £100 for taxes (on the 4th, 8th and 12th months).

**The Income Streams:** Zoe's financial life isn't all gloom and bills. She has two sources of income:

- Her College Salary: A government fund awards her advisor £1,900 each month, which trickles down to Zoe.
- Math tutor: Tutoring two kids in math brings an extra £150 per month.

This year, Zoe aims to spend at least £165 every month for fun activities. Recognizing that her monthly expenses fluctuate, she also plans to save a portion of her income each month. By doing so, she'll build up a reserve to cover the months with higher expenses. Zoe begins her second year with no savings from last year (£0).

(a) [3 points] Create a table that contains all individual monthly expenses and the corresponding monthly totals.

(b) [11 points] Formulate a linear programming problem that will help Zoe maximize the amount of money she spends for fun activities every month for the next 12 months. Describe in detail the decision variables, the objective and constraints in the model. (Hint: You may also consider that the amount she saves in month $t-1$ becomes income in month $t$).

(c) [11 points] Solve the optimization model using CVX or CvxPy and report the optimal solution. What is the total amount of money she will spend for fun in the following 12 months?

**Solution:**

(a) The summary of Zoe's monthly expenses is shown in Table 2.

| Month | Living exp. | Rent | Phone | Gas/Electr. | Transp. | Tax | Monthly exp. |
|---|---|---|---|---|---|---|---|
| 1 | 550 | 630 | | | 340 | | 1520 |
| 2 | 550 | 630 | 135 | | 340 | | 1655 |
| 3 | 550 | 630 | | | 340 | | 1520 |
| 4 | 550 | 630 | 135 | | 340 | 100 | 1755 |
| 5 | 550 | 630 | | | 340 | | 1520 |
| 6 | 550 | 630 | 135 | 850 | 340 | | 2505 |
| 7 | 550 | 630 | | | 340 | | 1520 |
| 8 | 550 | 630 | 135 | | 340 | 100 | 1755 |
| 9 | 550 | 630 | | | 340 | | 1520 |
| 10 | 550 | 630 | 135 | | 340 | | 1655 |
| 11 | 550 | 630 | | | 340 | | 1520 |
| 12 | 550 | 630 | 135 | 850 | 340 | 100 | 2605 |

Table 2: Itemized monthly expenses and total

Zoe has to be very careful with the management of her expenses. As can be easily seen in the above table, the expenses vary every month. If she spends all her money after she pays her monthly expenses she may not have enough money to cover her expenses for months 6 and 12. The only way to be able to ensure she can always cover her

expenses is to save some money every month and use it during those months that have high expenses.

We denote the known vector of the total monthly expenses as

$$E = (1520, 1655, 1520, 1755, 1520, 2505, 1520, 1755, 1520, 1655, 1520, 2605)^T$$

We will denote by $E_t$ the total monthly expenses of month $t$ (for example $E_3 = 1530$). This vector will be used in part (b) when we formulate the linear optimization problem.

(b) Zoe has to make two types of decisions every month.

- The first is how much money to spend for fun. Let us denote by $x_t$ that amount for month $t$. There are 12 such decision variables $(x_t, t = 1, 2, \ldots, 12)$.
- The second decision is how much money she needs to save every month. We denote by $s_t$ that amount for month $t$. There are 12 such decision variables $(s_t, t = 1, 2, \ldots, 12)$.

Her objective is to maximize the total amount of money she spends for fun for next 12 months:

$$\max \sum_{t=1}^{12} x_t$$

Her constraints ensure that she can always pay her monthly expenses with her total income and the money she she has been saving monthly.

For the $1^{st}$ month, she has to decide how much money to spend for fun, $x_1$, and how much to save, $s_1$. The sum of these two decision variables has to be equal to her total monthly income minus her total expenses for the 1st month:

$$x_1 + s_1 = 2050 - E_1$$

For all the remaining months (2 to 12) she has the following constraints

$$x_t + s_t \leq 2050 + s_{t-1} - E_t \;\; t = 2, 3, \ldots, 12$$

Note that the amount she has saved in month $t - 1$ is added to her income in the following month $t$.

In order to make sure she spends at least £165 per month for fun, she has to add the constraints in her model:

$$x_t \geq 165, \;\; t = 1, 2, \ldots, 12$$

And of course she has to include the nonnegativity constraints for the savings

$$s_t \geq 0, \;\; t = 1, 2, \ldots, 12$$

(c) You can use either Matlab or Python to solve the LP problem.

**Matlab implementation**

```
% Define your data
% include the montly salary (1900) and the tutoring money (150)
monthly_income = 2050;
min_money_for_fun = 165;
monthlyExpenses = [1520, 1655, 1520, 1755, 1520, 2505, 1520, 1755,
   1520, 1655, 1520, 2605];
T = 12;

% Define decision variables
cvx_begin
    variable money_for_fun(T) nonnegative
    variable savings(T) nonnegative

    % Objective: maximize the sum of money spent on fun
    maximize(sum(money_for_fun))

    % Constraints
    subject to
        % Expenses for the first month
        money_for_fun(1) + savings(1) <= monthly_income -
           monthlyExpenses(1)

        % Expenses for subsequent months
        for t = 2:T
             money_for_fun(t) + savings(t) - savings(t-1) <=
                monthly_income - monthlyExpenses(t)
        end

        % Spend at least 165 for fun every month
        money_for_fun >= min_money_for_fun
cvx_end

% Print results
fprintf('\nTotal amount spent for fun: %.2f\n', cvx_optval);
fprintf('\nMonthly amounts spent for fun:\n');
```

```
disp(money_for_fun);
fprintf('\nMonthly amounts saved:\n');
disp(savings);

% plots
planning_period = 1:12;
income = ones(1,12)*2050;
plot(planning_period, money_for_fun)
hold on
plot(planning_period, savings)
% plot(planning_period, monthlyExpenses)
% plot(planning_period, income)
xlabel('Time (months in a year)')
ylabel('Currency (GBP)')
legend('money for fun','savings')
% legend('money for fun','savings','monthly expenses','monthly income
    ')
```

## Python implementation

```
import numpy as np
import cvxpy as cp
import matplotlib.pyplot as plt

### Define your data ###
# include the montly salary (1900) and the tutoring money (150)
monthly_income = 2050
# minimum amount for fun
min_money_for_fun = 165
# agreegate monthly expneses
monthlyExpenses = np.array
    ([1520,1655,1520,1755,1520,2505,1520,1755,1520,1655,1520,2605])
# Time length of the planning period
T = 12


### Define the decision variables ###
# 'money_for_fun' and 'savings'
money_for_fun = cp.Variable(T, nonneg=True)
savings = cp.Variable(T, nonneg=True)
```

```
### Define your objective and constraints ###
# the sum of your monthly money you spend for fun stuff
obj = cp.sum(money_for_fun)
# expenses for the first month
constr = [money_for_fun[0] + savings[0] <= monthly_income -
    monthlyExpenses[0]]
# expenses for all subsequent months
constr += [money_for_fun[t] + savings[t] - savings[t-1] <=
    monthly_income - monthlyExpenses[t] for t in range(1,T)]
# should spend at least 165 for fun every month
constr += [money_for_fun >= min_money_for_fun]

### Define and solve your LP ###
prob = cp.Problem(cp.Maximize(obj), constr)
prob.solve()


### Print results ###
print('\nTotal amount spent for fun: \n', prob.value)
print('\nMonthly amounts spent for fun: \n', money_for_fun.value)
print('\nMonthly savings: \n', savings.value)


### Plot the money for fun and the savings
planning_period = range(T)
income_vec = [2050] * T # vector defining you monthly income
income = monthly_income * income_vec
plt.figure(figsize=(8,6))
plt.plot(planning_period, money_for_fun.value, label='money for fun')
plt.plot(planning_period, savings.value, label='savings')
plt.legend()
plt.xlabel('Time (months in a year)')
plt.ylabel('Currency (GBP)')
plt.show()
```

(d) The total amount of money Zoe can spend for fun in the next year is £3550.

Note that although we are solving an LP, the optimal solution is not unique. There are many ways for Zoe to save money every month so that she is able to cover her varying monthly expenses and use the remaining for fun activities. However the sum of the money she spends for fun is always £3550. This is what we found:

$$
\begin{aligned}
\texttt{money_for_fun} &= (354.91, 372.20, 355.29, 293.86, 240.39, 198.97, \\
&\quad 347.52, 334.57, 340.42, 297.27, 227.11, 187.43)^T \\
\texttt{savings} &= (175.08, 197.87, 372.58, 373.72, 663.33, 9.35, \\
&\quad 191.83, 152.24, 341.82, 439.54742.43, 0.0)^T
\end{aligned}
$$

Hence, most likely all of you have found a different solution, but all of you have found the same total amount. The reason is that the objective function of this LP depends only on the decision variables that are associated with the money Zoe spends for fun and not on the variables that correspond to the money she needs to save. Hence there are many ways to save which also correspond to many ways of spending for fun.

We also show the plot of the monthly savings and the money spent for fun is shown in Figure 2, although this is not required to get full grades in this problem.

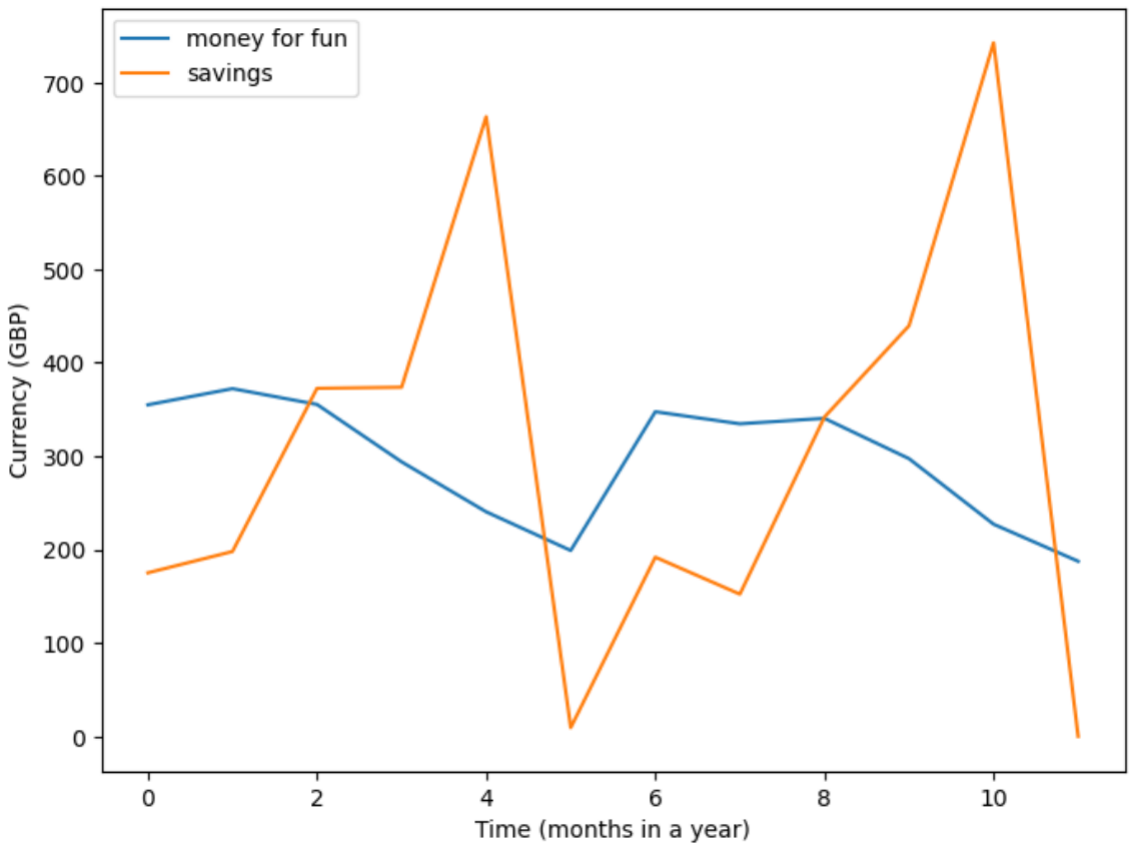


Figure 2: Zoe's monthly spending for fun activities and the savings she needs to make to cover her monthly expenses. Note that at time passes her spending for fun decreases until the 6th month and then increases for a few month until it starts decreasing again. Her savings mostly increase and on the 6th and 12th months they drop to 0 because she needs to cover the higher expenses of these months.

**Problem 4: Lyapunov functions and SDP**

Consider the linear dynamical system defined by the iterative formula:

$$x_{k+1} = Ax_k$$

where $A \in \mathbb{R}^{n\times n}$ is its transition matrix, and the quadratic function $V : \mathbb{R}^n \to \mathbb{R}$ with $V(x) = \frac{1}{2}x^T Qx$, for some symmetric matrix $Q \in \mathbb{R}^{n\times n}$.

(a) [6 points] Show that any matrix $Q$ that satisfies the following conditions

$$Q \succ 0 \text{ and } Q - A^T QA \succ 0$$

makes $V$ a Lyapunov function.

(b) [7 points] Use the above conditions to define an SDP problem whose solution gives you such a matrix $Q$.

(Hint: you can convert $\succ 0$ to $\succeq \epsilon I$ for some $\epsilon > 0$, say $\epsilon = 0.1$, and $I$ is the identity matrix in $\mathbb{R}^{n\times n}$.)

(c) [7 points] For the $2 \times 2$ matrix

$$A = \begin{bmatrix} 0.85 & 0.95, \\ -0.667 & 0.123 \end{bmatrix} \tag{10}$$

use CVX or CvxPy to solve the SDP (feasibility) problem you have defined in part (b).

(d) [5 points] Evaluate and plot 30 iterations of the dynamical system $x_{k+1} = Ax_k$ using as initial state $x_0 = (10, 10)^T$. These 30 pairs form the trajectory of the dynamical system. In a separate figure, evaluate and plot the values of the Lyapunov function you computed in (c) on the same trajectory of 30 pairs. Similarly, evaluate and plot the the function $L_2(x) = x^T x$ on the same trajectory. Which of the two functions ($V$ or $L_2$) decrease strictly? Could $L_2$ serve as a Lyapunov function? Briefly justify your answer.

**Solution**

(a) We will use Lyapunov's theorem for global stability and the three associated properties:

Property 1 (positivity): $V(0) = 0$ and $V(x) > 0, \forall x \in \mathbb{R}^n$ and $x \neq 0$. This property will be satisfied if

$$Q \succ 0. \tag{11}$$

because in this case $V(x) = \frac{1}{2}x^T Q x > 0, \forall x \in \mathbb{R}^n$ with $x \neq 0$, and $V(x) = 0$ only when $x = 0$.

Property 2 (radial unboundedness): $V(x) \to \infty$ as $\|x\| \to \infty$. This property is also satisfied if $Q \succ 0$.

Property 3 (strict decrease): $V(Ax) < V(x), \forall x \in \mathbb{R}^n$ and $x \neq 0$. Expanding the previous inequality we have

$$\frac{1}{2}(Ax)^T Q(Ax) < \frac{1}{2}x^T Q x \Rightarrow x^T A^T Q A x < x^T Q x \Rightarrow x^T (Q - A^T Q A) x > 0,$$

for all $x \in \mathbb{R}^n$ with $x \neq 0$. But this simply implies that

$$Q - A^T Q A \succ 0. \tag{12}$$

(b) Since we are looking for any matrix $Q$ that satisfies (11) and (12 we need to solve the following *feasibility* SDP problem:

$$\begin{aligned} \min \quad & 0 \\ s.t. \quad & Q - A^T Q A \succ 0 \\ & Q \succ 0 \end{aligned}$$

Note that the above SDP contains positive definite (PD) constraints. We can convert them to the positive semidefinite (PSD) constraints, by considering a small number $\epsilon > 0$. In that case we obtain the equivalent feasibility SDP

$$\begin{aligned} \min \quad & 0 \\ s.t. \quad & Q - A^T Q A - \epsilon I \succeq 0 \\ & Q - \epsilon I \succeq 0 \end{aligned}$$

(c) You can use either Matlab or Python to solve the specific instance of the SDP problem defined in (b).

**<u>Matlab implementation:</u>**

```
% Define the matrix A
A = [0.85, 0.95; -0.667, 0.123];

% Define the small positive number epsilon
epsilon = 0.1;

% define the SDP problem that compute a PSD matrix Q so that V is a
% Lyapunov function for the linear dynamic system.
% (Note the use of the matrix epsilon*I.)
cvx_begin
    variable Q(2,2) symmetric;

    minimize(0)
    subject to
        % ensure Q is PD
        Q - epsilon * eye(2,2) == semidefinite(2);

        % ensure Q - A'*Q*A is PD
        Q - A'*Q*A - epsilon * eye(2,2) == semidefinite(2);
cvx_end

disp('The matrix Q that makes V a Lyapunov function is: ')
disp(Q)

%%%% Plot the dynamical syste %%%%

% define the initial state
x0 = [10,10];

% generate the trajectoryby iterating the dynamical system
num_steps = 30;
trajectory = zeros(2, num_steps+1); % initialize trajectory matrix
trajectory(:,1) = x0;

for k = 2:num_steps
    x_next = A * trajectory(:,k-1);
    trajectory(:,k) = x_next;
end

% Extract x and y coordiates from the trajectory
x_coords = trajectory(1,:);
y_coords = trajectory(2,:);
```

```
% Plot the trajectory
figure;
plot(x_coords, y_coords, 'bo-', 'MarkerSize', 6, 'LineWidth', 1.5);
title('Dynamical System: $x_{k+1} = Ax_k$ (30 iterations)', '
   Interpreter', 'latex');
xlabel('$x^{(1)}$', 'Interpreter', 'latex');
ylabel('$x^{(2)}$', 'Interpreter', 'latex');
grid on;
axis equal;

%%%% Evaluate and plot the Lyapunov function %%%%
V = zeros(num_steps+1);
for k = 1:num_steps
    V(k) = trajectory(:,k)'*Q*trajectory(:,k);
end

figure;
plot(V(1:31), 'bo-', 'MarkerSize', 6, 'LineWidth', 1.5);
title('Lyapunov function (30 iterations)', 'Interpreter', 'latex');
xlabel('$k$', 'Interpreter', 'latex');
ylabel('$V(x_k)$', 'Interpreter', 'latex');

%%%% Evaluate and plot the L2 function %%%%
L2 = zeros(num_steps+1);
for k = 1:num_steps
    L2(k) = trajectory(:,k)'*trajectory(:,k);
end

figure;
plot(L2(1:31), 'ro-', 'MarkerSize', 6, 'LineWidth', 1.5);
title('L2-norm (30 iterations)', 'Interpreter', 'latex');
xlabel('$k$', 'Interpreter', 'latex');
ylabel('$L2(x_k)$', 'Interpreter', 'latex');
```

**Python implementation:**

```
import numpy as np
import matplotlib.pyplot as plt
import cvxpy as cp

# Define the matrix A
```

```python
A = np.array([ [0.85,0.95], [-0.667,0.123] ])

# Initial condition (choose any starting point)
x0 = np.array([10, 10])

# Generate the trajectory by iterating the dynamical system
num_steps = 35
trajectory = [x0]
for _ in range(num_steps):
    x_next = np.dot(A, trajectory[-1])
    trajectory.append(x_next)

# Extract x and y coordinates from the trajectory
x1_coords, x2_coords = zip(*trajectory)

# Plot the trajectory
plt.figure(figsize=(6, 6))
plt.plot(x1_coords, x2_coords, marker='o', linestyle='-', color='b')
plt.title("Dynamical System: $x_{k+1} = Ax_k$")
plt.xlabel("$x^{(1)}$")
plt.ylabel("$x^{(2)}$")
plt.grid(True)
plt.show()

def solve_lyapunov_sdp(A, verbose=True):
    """solves the feasibility problem P > 0, (1-g)*P-A'PA > 0"""
    n, _ = A.shape
    P = cp.Variable(shape=(n,n), symmetric=True)
    I = np.eye(n)
    epsilon = 1

    obj = cp.Minimize(0)
    constr = [P >> epsilon * I,  P - A.T @ P @ A >> epsilon * I]

    prob = cp.Problem(obj,constr)
    prob.solve(verbose=verbose)

    return prob.value, P.value

g = 0
p, P = solve_lyapunov_sdp(A)
print(p, P)
```

```
#P = np.array([[8.419, 3.636],[3.636, 12.109]])

V_traj = []
for i in range(len(trajectory)):
    Px = np.dot(P,trajectory[i])
    xPx = np.dot(trajectory[i].T,Px)
    V_traj.append(xPx)

k = range(len(V_traj))
plt.plot(k, V_traj, marker='o')
plt.xlabel('k')
plt.ylabel('$V(x_k)$')
plt.title('Lyapunov function convergence (30 iterates)')
plt.show()

L2_traj = []
for i in range(len(trajectory)):
    L2_traj_k = np.linalg.norm(trajectory[i], 2)
    L2_traj.append(L2_traj_k)

k = range(len(V_traj))
plt.plot(k, L2_traj, marker='*')
plt.xlabel('k')
plt.ylabel('$L2(x_k)$')
plt.title('L2-norm convergence (30 iterates)')
plt.show()
```

It is evident from Figure 5 that the function $L2$ is not strictly decreasing at every time point. Hence it cannot be a Lyapunov function, because Property 3 will be violated.

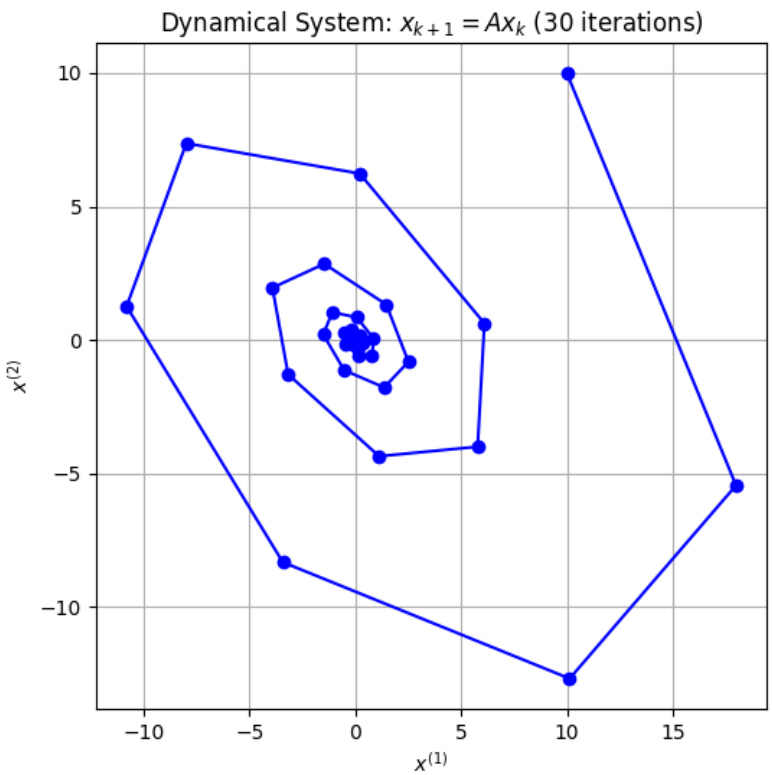


Figure 3: Plot of the trajectory of the dynamical system (30 points)

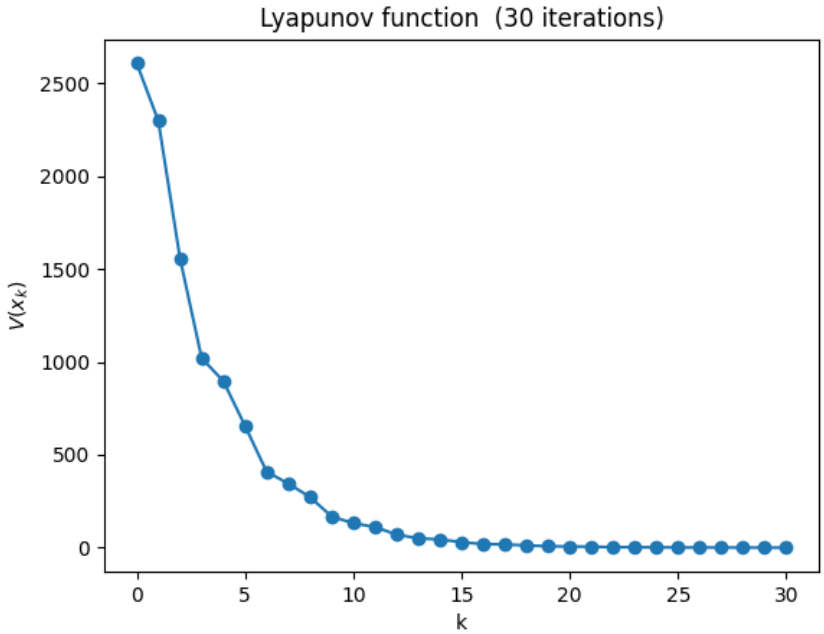


Figure 4: Plot of the Lyapunov function on the same trajectory (30 points)

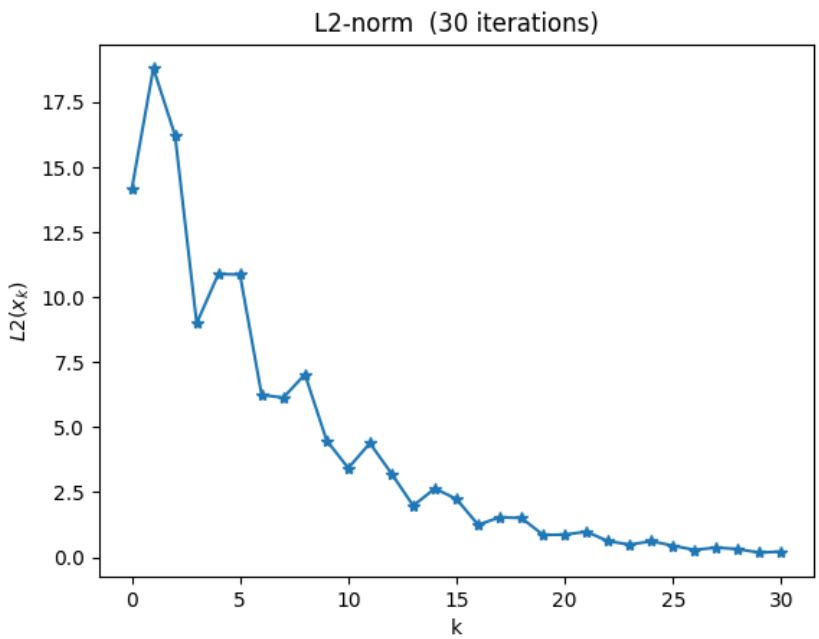


Figure 5: Plot of the L2 function on the same trajectory (30 points)